TITLE: View Aligned

AIM: To develop a web-based GUI application that automates the bioinformatics pipeline for aligning raw sequencing reads to a reference genome and provide real-time visualization of the results.

THEORY:

The current landscape of bioinformatics often feels fragmented, requiring researchers to navigate a disjointed ecosystem of command-line tools and heavy desktop applications. Traditionally, a biologist must first master complex terminal commands to run an aligner like BWA-MEM, then manually handle intermediate file conversions and indexing using SAMtools, before finally loading the output into a separate visualization tool like the Integrative Genomics Viewer (IGV). This "tool-switching" is not just annoying; it creates significant friction in the research workflow, increasing the likelihood of manual errors and data-handling oversights at every step.

The "VIEW ALIGNED" project addresses this fragmentation by unifying these disparate stages into a single, cohesive workflow powered by the Julia programming language. By automating the backend pipeline, from raw read alignment to real-time visualization. The project eliminates the need for users to touch the command line or manage complex file paths. This integration ensures that the moment the alignment is completed by BWA-MEM, the results are immediately ready for inspection, bridging the gap between raw data and biological insight.

At the heart of the alignment process is BWA-MEM, a powerful algorithm designed to map low-divergence sequence reads against a large reference genome. While highly efficient, its native command-line interface can be intimidating for those without extensive computational training. "VIEW ALIGNED" wraps this industrial-strength aligner in a user-friendly GUI, allowing anyone to perform high-quality sequence mapping by simply uploading their FASTA and FASTQ files.

Once the data is aligned, the Integrative Genomics Viewer (IGV.js) integration provides the critical visual context needed to interpret the results. IGV allows researchers to interactively explore the genomic landscape, zooming in to see individual base pairs or zooming out to view broad coverage patterns. By embedding this viewer directly into the application, "VIEW ALIGNED" provides an "all-in-one" environment where mapping statistics and visual evidence are presented side-by-side for immediate scientific validation.

Ultimately, the transition to a web-based Graphical User Interface (GUI) democratizes complex bioinformatics. It transforms a process that previously required specialized software installations and coding knowledge into a streamlined experience accessible through a standard web browser. This project demonstrates that by centralizing tools like BWA-MEM and IGV into one interface, we can empower a broader range of scientists, from biologists to bioinformaticians to focus on their research questions rather than their software hurdles.

Programming Languages:

Julia:

Julia is a high-performance open source programming language designed for scientific and numerical computing. Created to solve the "two-language problem," Julia combines the ease and expressiveness of languages like Python and R with performance comparable to C/C++. Released in 2012, Julia has grown significantly and is now used at more than 1,500 universities and 10,000 companies worldwide.

HTML:

HTML (HyperText Markup Language) is the most basic building block of the Web. It defines the meaning and structure of web content. Other technologies besides HTML are generally used to describe a web page's appearance/presentation (CSS) or functionality/behavior (JavaScript).

"Hypertext" refers to links that connect web pages to one another, either within a single website or between websites. Links are a fundamental aspect of the Web. By uploading content to the Internet and linking it to pages created by other people, you become an active participant in the World Wide Web.

Bash:

Bash is a command-line interpreter or Unix Shell and it is widely used in GNU/Linux Operating System. It is written by Brian Jhan Fox. It is used as a default login shell for most Linux distributions. Scripting is used to automate the execution of the tasks so that humans do not need to perform them individually. Bash scripting is a great way to automate different types of tasks in a system. Developers can avoid doing repetitive tasks using bash scripting.

JavaScript:

JavaScript (often abbreviated as JS) is a multi-paradigm programming language that is most famously known as the scripting language for Web pages. While languages like C++ or Java must be "compiled" into machine code before they run, JavaScript is typically interpreted by a browser's engine (like Google's V8 or Mozilla's SpiderMonkey) as it is being read.

OBJECTIVES:

- To implement an automated backend pipeline using Julia to execute NGS tools (BWA and Samtools).

- To create a dynamic web interface using HTTP.jl for file uploads (FASTA and FASTQ).

- To automate the indexing and sorting of genomic data to ensure compatibility with downstream visualization.

- To integrate IGV.js (Integrative Genomics Viewer) for interactive visualization of aligned reads.

• To generate and display mapping statistics to provide scientific validation of the alignment quality.

• To implement locus detection from FASTA headers to simplify the user experience.

EXPECTED OUTPUT:

BWA-MEM working in the backend for producing alignment.

Web Portal: A clean, centered home screen for uploading .fa and .fastq files.

Scientific Dashboard, i.e, a results page showing:

- The detected Locus/Sequence ID.
- A "Mapping Quality Report" with metrics like total reads, mapping percentage, and pairing status.
- An interactive IGV track where reads are visible against the reference sequence.

OBSERVED RESULTS:

- The backend successfully executed the BWA-MEM alignment pipeline to produce sequence alignments.
- A Graphical User Interface (GUI) was established that allowed for smooth navigation across distinct pages.
- Mapping statistics and a scientific quality control report were generated to validate the alignment.
- Real-time genomic visualization was achieved through the integration of IGV.js, enabling interactive data exploration.
- Specific visualization features were verified as functional, including a cursor guide, three-frame translation options, and customizable track colors.
- The system correctly generated and allowed for the download of standard bioinformatics output files, specifically .bam, .bai, and .svg formats

WORKFLOW:

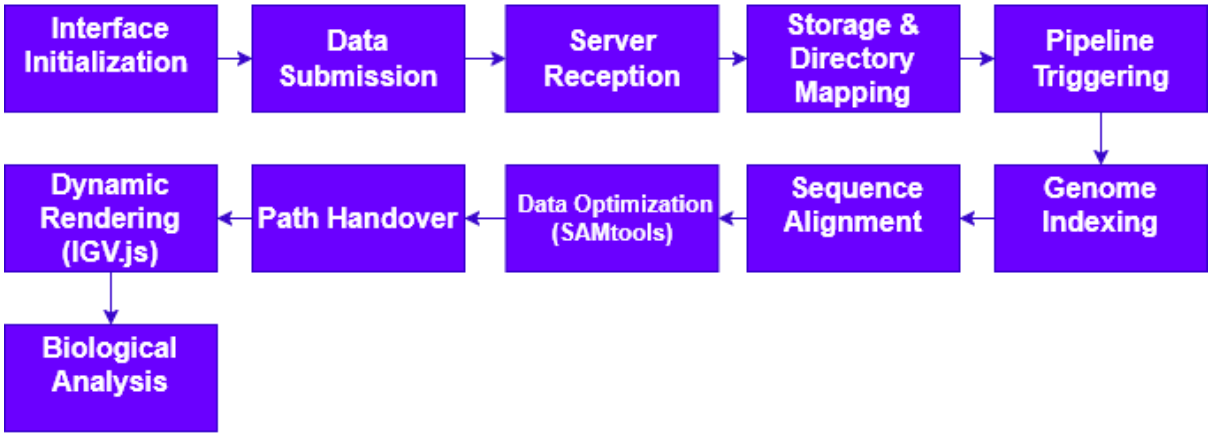

Fig 1: Flowchart of the program

CODE:

```
using HTTP
using Sockets
using Dates
BASE_DIR   = "/mnt/d/Unnati/Msc_II/SN/miniproj/alignment_app"
UPLOAD_DIR = joinpath(BASE_DIR, "uploads")
OUT_DIR    = joinpath(BASE_DIR, "output")

mkpath(UPLOAD_DIR)
mkpath(OUT_DIR)

function run_pipeline(ref_path, reads_path, output_prefix)
    bam_out = output_prefix * ".sorted.bam"
    stats_out = output_prefix * ".stats.txt"

    try
        # Step A: Indexing reference
```

```
        if !isfile(ref_path * ".bwt")
            @info "Indexing reference..."
            run(`bwa index "$ref_path"`)
        end
        if !isfile(ref_path * ".fai")
            run(`samtools faidx "$ref_path"`)
        end

        # Step B: Align & Sort
        @info "Running Alignment..."
        pipeline_cmd = pipeline(
            `bwa mem -t 4 "$ref_path" "$reads_path"`,
            `samtools view -Sb -`,
            `samtools sort -o "$bam_out"`
        )
        run(pipeline_cmd)

        run(`samtools index "$bam_out"`)
        @info "Generating Mapping Stats..."
        stats_data = read(`samtools flagstat "$bam_out"`, String)
        write(stats_out, stats_data)

        return bam_out, stats_data
    catch e
        @error "Pipeline Error" exception=e
        return nothing, nothing
    end
end
```

```
function get_fasta_locus(ref_path)
    for line in eachline(ref_path)
        if startswith(line, ">")
            return split(replace(line, ">" => ""))[1]
        end
    end
    return "all"
end

function viewer_html(bam_filename, ref_filename, locus_name, stats_text)
    formatted_stats = replace(stats_text, "\n" => "<br>")

    return """
    <html>
    <head>
        <title>VIEW ALIGNED</title>
        <script src="https://cdn.jsdelivr.net/npm/igv@2.15.5/dist/igv.min.js"></script>
        <style>
            body { font-family: 'Segoe UI', sans-serif; padding: 30px; background-color: #f4f7f6; color: #333; }
            .container { max-width: 1100px; margin: auto; background: white; padding: 25px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
            .header-info { display: flex; justify-content: space-between; align-items: center; border-bottom: 2px solid #eee; padding-bottom: 15px; margin-bottom: 20px; }
            .stats-box { background: #f8f9fa; padding: 20px; border-left: 5px solid #28a745; font-family: 'Courier New', monospace; font-size: 0.85em; margin-bottom: 25px; overflow-x: auto; }
            h2 { margin: 0; color: #2c3e50; }
            #igv-div { border: 1px solid #ddd; border-radius: 8px; background: white; }
        </style>
    </head>
```

```
<body>
    <div class="container">
        <div class="header-info">
            <h2> Alignment Analysis</h2>
            <div style="text-align: right; font-size: 0.9em; color: #666;">
                <strong>Ref:</strong> $ref_filename <br>
                <strong>Locus:</strong> $locus_name
            </div>
        </div>

        <div class="stats-box">
            <strong> Mapping Quality Report (samtools flagstat):</strong><br><br>
            $formatted_stats
        </div>

        <div id="igv-div"></div>
    </div>

    <script type="text/javascript">
        var igvDiv = document.getElementById("igv-div");
        var options = {
            reference: {
                id: "$locus_name",
                fastaURL: "/download_ref/$ref_filename",
                indexURL: "/download_ref/$ref_filename.fai"
            },
            locus: "$locus_name",
            tracks: [
                {
```

```
                name: "Aligned Reads",
                url: "/download/$bam_filename",
                indexURL: "/download/$bam_filename.bai",
                format: "bam",
                type: "alignment",
                displayMode: "EXPANDED",
                height: 400
            }
        ]
    };
    igv.createBrowser(igvDiv, options);
  </script>
</body>
</html>
"""
end

@info "Server starting at http://localhost:8001"
HTTP.serve("0.0.0.0", 8001) do req::HTTP.Request

    if startswith(req.target, "/download/")
        fname = replace(req.target, "/download/" => "")
        fpath = joinpath(OUT_DIR, fname)
        isfile(fpath) && return HTTP.Response(200, ["Access-Control-Allow-Origin" => "*"],
read(fpath))
    end

    if startswith(req.target, "/download_ref/")
        fname = replace(req.target, "/download_ref/" => "")
```

```
        fpath = joinpath(UPLOAD_DIR, fname)
        isfile(fpath) && return HTTP.Response(200, ["Access-Control-Allow-Origin" => "*"], read(fpath))
    end

    if req.method == "GET" && req.target == "/"
        return HTTP.Response(200, ["Content-Type" => "text/html"],
            """
            <html>
            <head>
                <title>VI</title>
                <style>
                    body { font-family: 'Segoe UI', sans-serif; background: #e9ecef; display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; }
                    .card { background: white; padding: 40px; border-radius: 15px; box-shadow: 0 10px 25px rgba(0,0,0,0.1); width: 400px; text-align: center; }
                    h2 { color: #333; margin-bottom: 10px; }
                    p { color: #777; margin-bottom: 30px; font-size: 0.9em; }
                    .group { text-align: left; margin-bottom: 20px; }
                    label { display: block; margin-bottom: 8px; font-weight: 600; color: #495057; }
                    input[type='file'] { width: 100%; padding: 10px; border: 1px solid #ced4da; border-radius: 5px; box-sizing: border-box; }
                    input[type='submit'] { background: #007bff; color: white; border: none; padding: 14px; width: 100%; border-radius: 5px; font-size: 16px; font-weight: bold; cursor: pointer; transition: 0.3s; }
                    input[type='submit']:hover { background: #0056b3; }
                </style>
            </head>
            <body>
                <div class='card'>
                    <h2>VIEW ALIGNED</h2>
```

```
            <p>A GUI powered integrated tool, that allows both alignment and visualization of your reads.</p>
              <p>A true mate for Biologists turned Bioinformaticians!<p>
            <form method='POST' action='/align' enctype='multipart/form-data' onsubmit="this.btn.value='Aligning...'; this.btn.disabled=true;">
              <div class='group'>
                <label>Reference FASTA (.fa)</label>
                <input type='file' name='ref' required>
              </div>
              <div class='group'>
                <label>Read FASTQ (.fastq)</label>
                <input type='file' name='reads' required>
              </div>
              <input type='submit' name='btn' value='Align & View Results'>
            </form>
          </div>
        </body>
        </html>
        """)
    end

    if req.method == "POST" && req.target == "/align"
      try
        parts = HTTP.parse_multipart_form(req)
        local ref_path, reads_path, ref_filename

        for p in parts
          if p.name == "ref"
            ref_filename = p.filename
            ref_path = joinpath(UPLOAD_DIR, p.filename)
```

```
            write(ref_path, p.data)
        elseif p.name == "reads"
            reads_path = joinpath(UPLOAD_DIR, p.filename)
            write(reads_path, p.data)
        end
    end

    locus_name = get_fasta_locus(ref_path)
    timestamp = Dates.format(now(), "HHmmss")
    bam_name = "result_$timestamp.sorted.bam"
    out_prefix = joinpath(OUT_DIR, "result_$timestamp")

    bam_path, stats_data = run_pipeline(ref_path, reads_path, out_prefix)

    if bam_path !== nothing
        return HTTP.Response(200, ["Content-Type" => "text/html"],
            viewer_html(bam_name, ref_filename, locus_name, stats_data))
    end
  catch e
    return HTTP.Response(500, "Server Error: $e")
  end
end

return HTTP.Response(404, "Not Found")
end
```

MODULES

```
using HTTP
using Sockets
```

```
using Dates
```

Module 1: CONFIGURATION

```
# Ensure these paths exist on your system
BASE_DIR   = "/mnt/d/Unnati/Msc_II/SN/miniproj/alignment_app"
UPLOAD_DIR = joinpath(BASE_DIR, "uploads")
OUT_DIR    = joinpath(BASE_DIR, "output")

mkpath(UPLOAD_DIR)
mkpath(OUT_DIR)
```

Module 2: BIOINFORMATICS PIPELINE

```
function run_pipeline(ref_path, reads_path, output_prefix)
    bam_out = output_prefix * ".sorted.bam"
    stats_out = output_prefix * ".stats.txt"

    try
        # Step A: Indexing reference
        if !isfile(ref_path * ".bwt")
            @info "Indexing reference..."
            run(`bwa index "$ref_path"`)
        end
        if !isfile(ref_path * ".fai")
            run(`samtools faidx "$ref_path"`)
        end

        # Step B: Align & Sort
        @info "Running Alignment..."
        pipeline_cmd = pipeline(
```

```
        `bwa mem -t 4 "$ref_path" "$reads_path"`,
        `samtools view -Sb -`,
        `samtools sort -o "$bam_out"`
    )
    run(pipeline_cmd)

    # Step C: Index the BAM
    run(`samtools index "$bam_out"`)

    # Step D: Generate Statistics
    @info "Generating Mapping Stats..."
    stats_data = read(`samtools flagstat "$bam_out"`, String)
    write(stats_out, stats_data)

    return bam_out, stats_data
  catch e
    @error "Pipeline Error" exception=e
    return nothing, nothing
  end
end

# Helper to get the first sequence name from FASTA
function get_fasta_locus(ref_path)
  for line in eachline(ref_path)
    if startswith(line, ">")
      return split(replace(line, ">" => ""))[1]
    end
  end
  return "all"
```

```
end
```

Module 3: VIEWER HTML (Results Page)

```
function viewer_html(bam_filename, ref_filename, locus_name, stats_text)
    formatted_stats = replace(stats_text, "\n" => "<br>")


    return """
    <html>
    <head>
        <title>VIEW ALIGNED</title>
        <script src="https://cdn.jsdelivr.net/npm/igv@2.15.5/dist/igv.min.js"></script>
        <style>
            body { font-family: 'Segoe UI', sans-serif; padding: 30px; background-color: #f4f7f6; color: #333; }
            .container { max-width: 1100px; margin: auto; background: white; padding: 25px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
            .header-info { display: flex; justify-content: space-between; align-items: center; border-bottom: 2px solid #eee; padding-bottom: 15px; margin-bottom: 20px; }
            .stats-box { background: #f8f9fa; padding: 20px; border-left: 5px solid #28a745; font-family: 'Courier New', monospace; font-size: 0.85em; margin-bottom: 25px; overflow-x: auto; }
            h2 { margin: 0; color: #2c3e50; }
            #igv-div { border: 1px solid #ddd; border-radius: 8px; background: white; }
        </style>
    </head>
    <body>
        <div class="container">
            <div class="header-info">
                <h2> Alignment Analysis</h2>
                <div style="text-align: right; font-size: 0.9em; color: #666;">
                    <strong>Ref:</strong> $ref_filename <br>
```

```
            <strong>Locus:</strong> $locus_name
        </div>
    </div>

    <div class="stats-box">
        <strong> Mapping Quality Report (samtools flagstat):</strong><br><br>
        $formatted_stats
    </div>

    <div id="igv-div"></div>
</div>

<script type="text/javascript">
    var igvDiv = document.getElementById("igv-div");
    var options = {
        reference: {
            id: "$locus_name",
            fastaURL: "/download_ref/$ref_filename",
            indexURL: "/download_ref/$ref_filename.fai"
        },
        locus: "$locus_name",
        tracks: [
            {
                name: "Aligned Reads",
                url: "/download/$bam_filename",
                indexURL: "/download/$bam_filename.bai",
                format: "bam",
                type: "alignment",
                displayMode: "EXPANDED",
```

```
                height: 400
            }
        ]
    };
    igv.createBrowser(igvDiv, options);
  </script>
</body>
</html>
"""
end
```

Module 4: SERVER LOGIC

```
@info "Server starting at http://localhost:8001"
HTTP.serve("0.0.0.0", 8001) do req::HTTP.Request

    # File Download Routes
    if startswith(req.target, "/download/")
        fname = replace(req.target, "/download/" => "")
        fpath = joinpath(OUT_DIR, fname)
        isfile(fpath) && return HTTP.Response(200, ["Access-Control-Allow-Origin" => "*"],
read(fpath))
    end

    if startswith(req.target, "/download_ref/")
        fname = replace(req.target, "/download_ref/" => "")
        fpath = joinpath(UPLOAD_DIR, fname)
        isfile(fpath) && return HTTP.Response(200, ["Access-Control-Allow-Origin" => "*"],
read(fpath))
    end
```

```
# GET: Form Page (Centered UI)
if req.method == "GET" && req.target == "/"
  return HTTP.Response(200, ["Content-Type" => "text/html"],
    """
    <html>
    <head>
      <title>VI</title>
      <style>
        body { font-family: 'Segoe UI', sans-serif; background: #e9ecef; display: flex; justify-content: center; align-items: center; height: 100vh; margin: 0; }
        .card { background: white; padding: 40px; border-radius: 15px; box-shadow: 0 10px 25px rgba(0,0,0,0.1); width: 400px; text-align: center; }
        h2 { color: #333; margin-bottom: 10px; }
        p { color: #777; margin-bottom: 30px; font-size: 0.9em; }
        .group { text-align: left; margin-bottom: 20px; }
        label { display: block; margin-bottom: 8px; font-weight: 600; color: #495057; }
        input[type='file'] { width: 100%; padding: 10px; border: 1px solid #ced4da; border-radius: 5px; box-sizing: border-box; }
        input[type='submit'] { background: #007bff; color: white; border: none; padding: 14px; width: 100%; border-radius: 5px; font-size: 16px; font-weight: bold; cursor: pointer; transition: 0.3s; }
        input[type='submit']:hover { background: #0056b3; }
      </style>
    </head>
    <body>
      <div class='card'>
        <h2>VIEW ALIGNED</h2>
        <p>A GUI powered integrated tool, that allows both alignment and visualization of your reads.</p>
          <p>A true mate for Biologists turned Bioinformaticians!<p>
```

```
            <form method='POST' action='/align' enctype='multipart/form-data' onsubmit="this.btn.value='Aligning...'; this.btn.disabled=true;">
                <div class='group'>
                    <label>Reference FASTA (.fa)</label>
                    <input type='file' name='ref' required>
                </div>
                <div class='group'>
                    <label>Read FASTQ (.fastq)</label>
                    <input type='file' name='reads' required>
                </div>
                <input type='submit' name='btn' value='Align & View Results'>
            </form>
        </div>
    </body>
    </html>
    """)
end

# POST: Run Pipeline
if req.method == "POST" && req.target == "/align"
    try
        parts = HTTP.parse_multipart_form(req)
        local ref_path, reads_path, ref_filename

        for p in parts
            if p.name == "ref"
                ref_filename = p.filename
                ref_path = joinpath(UPLOAD_DIR, p.filename)
                write(ref_path, p.data)
```

```
            elseif p.name == "reads"
                reads_path = joinpath(UPLOAD_DIR, p.filename)
                write(reads_path, p.data)
            end
        end

        locus_name = get_fasta_locus(ref_path)
        timestamp = Dates.format(now(), "HHmmss")
        bam_name = "result_$timestamp.sorted.bam"
        out_prefix = joinpath(OUT_DIR, "result_$timestamp")

        bam_path, stats_data = run_pipeline(ref_path, reads_path, out_prefix)

        if bam_path !== nothing
            return HTTP.Response(200, ["Content-Type" => "text/html"],
                viewer_html(bam_name, ref_filename, locus_name, stats_data))
        end
    catch e
        return HTTP.Response(500, "Server Error: $e")
    end
  end

  return HTTP.Response(404, "Not Found")
end
```

RESULTS:

```
julia> include("/mnt/d/Unnati/Msc_II/SN/miniproj/alignment_app/webalignment1.jl")
[ Info: Server starting at http://localhost:8001
[ Info: Listening on: 0.0.0.0:8001, thread id: 1
[ Info: Running Alignment...
[M::bwa_idx_load_from_disk] read 0 ALT contigs
[M::process] read 1 sequences (66 bp)...
[M::mem_process_seqs] Processed 1 reads in 0.005 CPU sec, 0.005 real sec
[main] Version: 0.7.17-r1188
[main] CMD: bwa mem -t 4 /mnt/d/Unnati/Msc_II/SN/miniproj/alignment_app/uploads/genome1.fa /mnt/d/Unnati/Msc_II/SN/minip
roj/alignment_app/uploads/reads1.fastq
[main] Real time: 0.064 sec; CPU: 0.032 sec
[ Info: Generating Mapping Stats...
```

Fig 2: Backend Execution of the BWA-MEM Alignment Pipeline

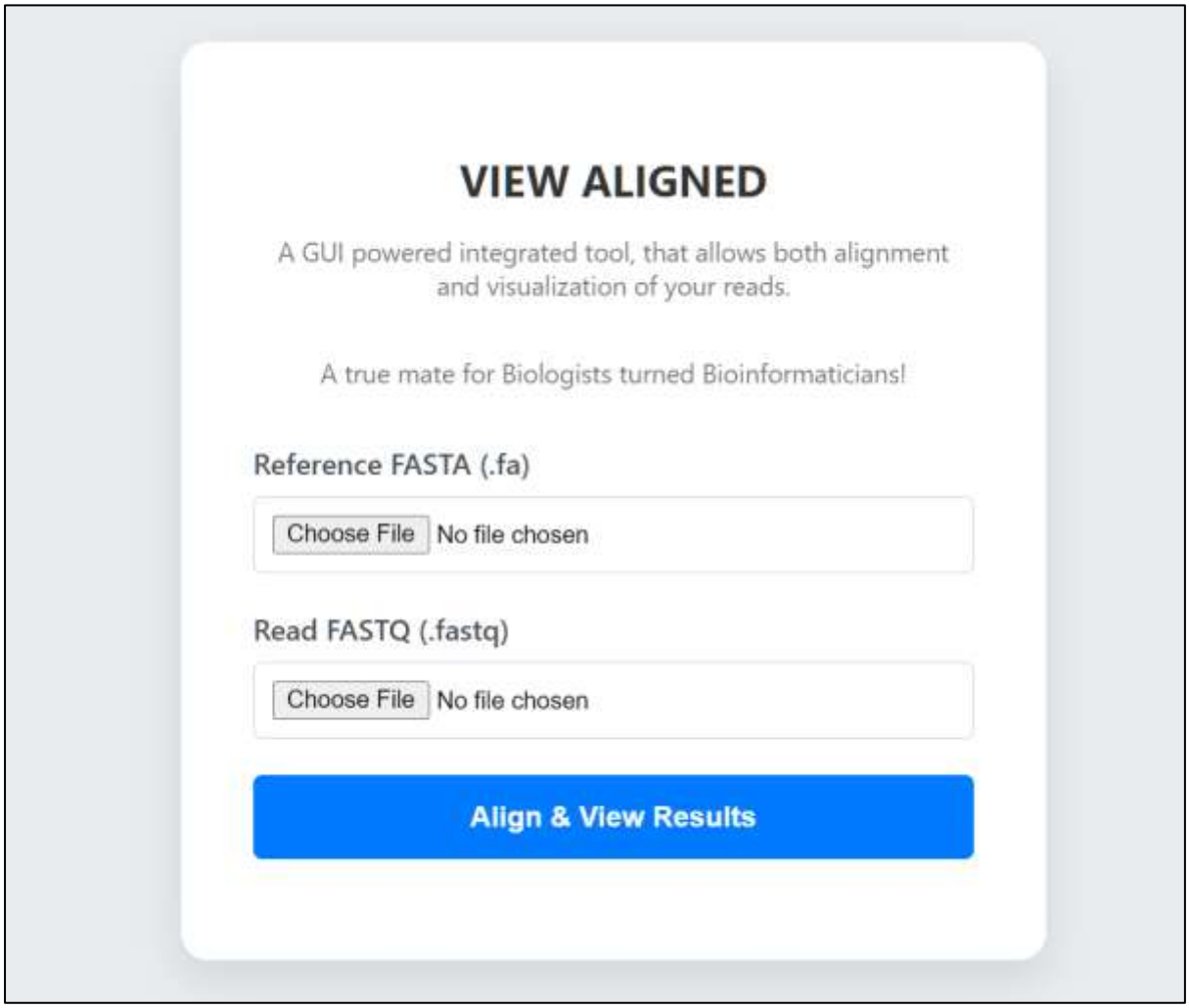

Fig 3: Graphical User Interface for Sequence Upload

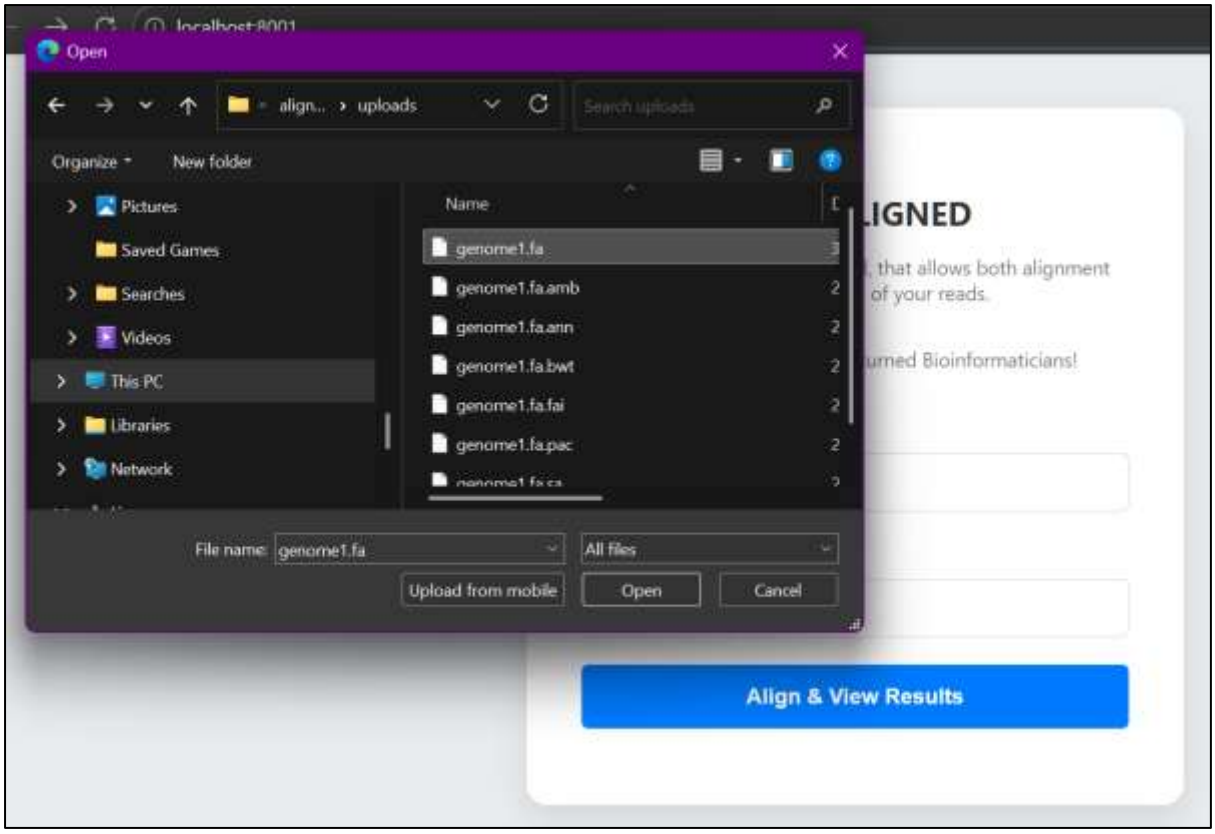

Fig 4: Selection of the reference genome and the reads

Fig 5: Mapping Statistics and Scientific Quality Control Report

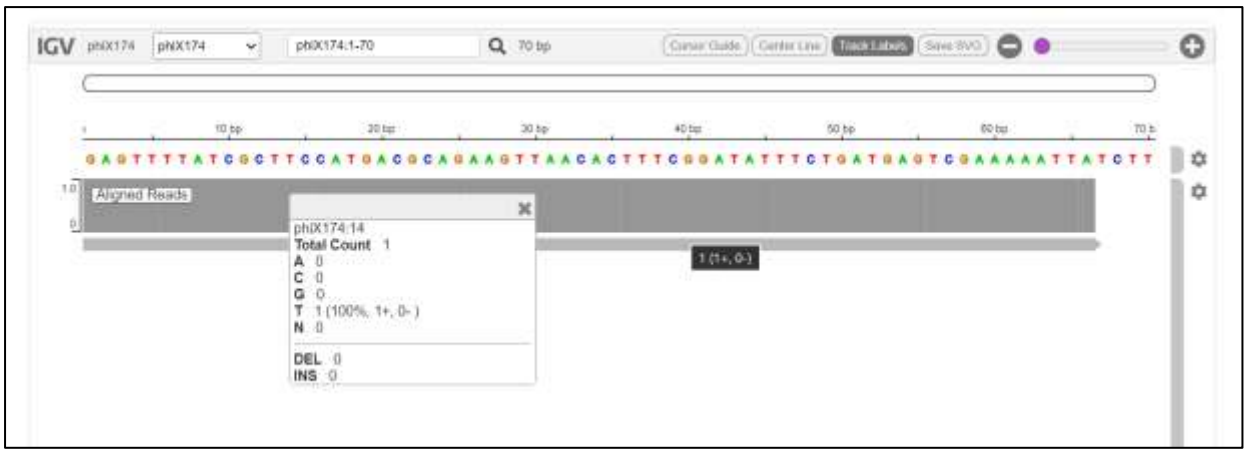

Fig 6: Genomic Visualization via IGV.js Integration

Fig 7: Download of the .svg file

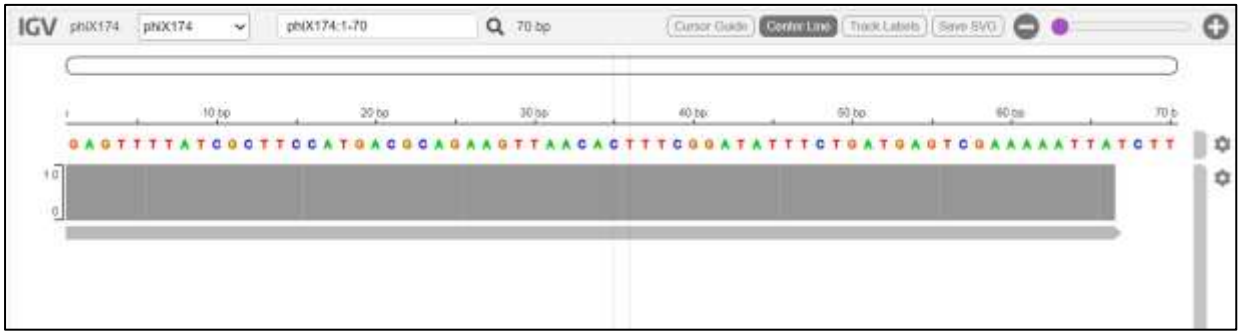

Fig 8: Center line for the visualization of the alignment

Fig 9: Option enabled for cursor guide

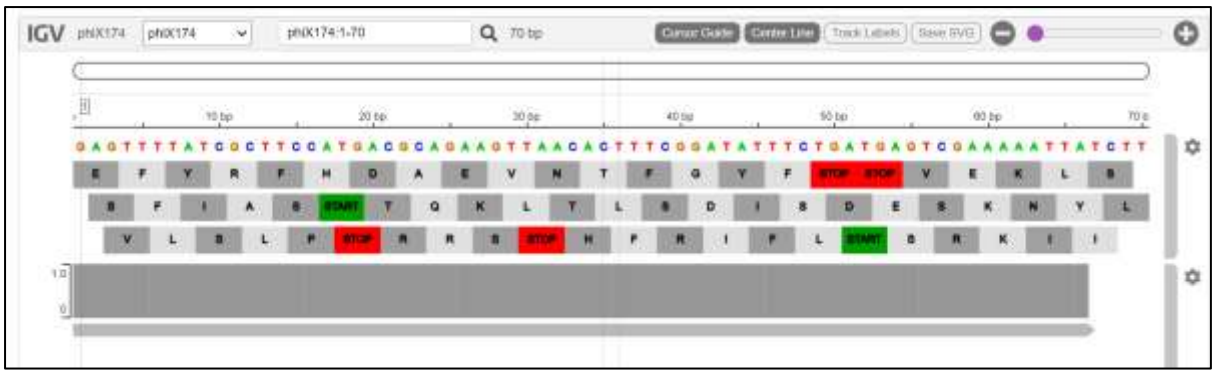

Fig 10: Option enables for 3 frame translation

Fig 11: Option enabled to select track colour

result_$timestamp.sorted.bam	30-12-2025 19:12	BAM File	1 KB
result_$timestamp.sorted.bam.bai	30-12-2025 19:12	BAI File	1 KB
result_1312101.sorted.bam	30-12-2025 19:16	BAM File	1 KB
result_1312101.sorted.bam.bai	30-12-2025 19:16	BAI File	1 KB
result_1401615.sorted.bam	01-01-2026 19:52	BAM File	1 KB
result_1401615.sorted.bam.bai	01-01-2026 19:52	BAI File	1 KB
result_1401615.stats	01-01-2026 19:52	Text Document	1 KB
result_1412759.sorted.bam	30-12-2025 19:42	BAM File	1 KB
result_1412759.sorted.bam.bai	30-12-2025 19:42	BAI File	1 KB

Fig 12: bam and bai file generated by BWA-MEM and SamTools

CONCLUSION:

A fully functional tool, 'VIEW ALIGNED,' was successfully developed. The project demonstrates the power of Julia in orchestrating complex bioinformatics workflows, bridging the gap between raw command-line tools and user-friendly web interfaces. The tool accurately processes raw sequencing data and provides biologically meaningful visualizations in real-time.

FUTURE SCOPE:

• Multithreading: Expand the bwa mem -t parameter to allow for faster processing of large-scale human datasets.

• Variant Calling: Integrate tools like GATK or BCFtools to identify mutations directly from the interface.

• Cloud Integration: Deploy the application via Docker containers for cross-platform portability in research environments.

• Multi-Track Support: Allow users to upload and compare multiple BAM files (e.g., Case vs. Control) in the same window.

REFERENCES:

1. Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*.
2. Li, H. (2013). Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv preprint*.
3. Danecek, P., et al. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*.
4. Robinson, J. T., et al. (2011). Integrative Genomics Viewer. *Nature Biotechnology*.
5. GeeksforGeeks. (2025, July 23). *Bash Scripting Introduction to Bash and bash Scripting*. GeeksforGeeks. https://www.geeksforgeeks.org/linux-unix/bash-scripting-introduction-to-bash-and-bash-scripting/
6. *HTML: HyperText Markup Language | MDN*. (n.d.). https://developer.mozilla.org/en-US/docs/Web/HTML
7. Haverbeke, M. (2018). *Eloquent JavaScript: A modern introduction to programming* (3rd ed.).